

LayerZero Console Solana

Security Assessment

October 13th, 2026 — Prepared by OtterSec

Akash Gurugunti sud0u53r.ak@osec.io

Mingxuan Yang ymxf00l@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **10**

Vulnerabilities **11**

OS-LCS-ADV-00 | Unsafe Admin Reinitialization 12

OS-LCS-ADV-01 | EVM–Solana Peer Existence Semantics Mismatch 13

OS-LCS-ADV-02 | Default Gate Omits Thaw Metas 14

General Findings **15**

OS-LCS-SUG-00 | Stale Hook Cache Blocks Inbound Planning 17

OS-LCS-SUG-01 | OFTFeeDetail Type Mismatch in Docs 18

OS-LCS-SUG-02 | Inconsistent Idempotency Across Admin Setters 19

OS-LCS-SUG-03 | Missing Transfer Invocation Guard 20

OS-LCS-SUG-04 | Inconsistencies in Documentation and Macro Semantics 21

OS-LCS-SUG-05 | Insufficient Validation and Type Safety 23

OS-LCS-SUG-06 | Code Duplication 26

OS-LCS-SUG-07 | Unnecessary System Program Account in RBAC 28

OS-LCS-SUG-08 | Cross-Domain Instruction Attribute Conflict 29

OS-LCS-SUG-09 | Unnecessary Event Emissions 30

Appendices

Vulnerability Rating Scale **31**

Procedure **32**

© 2026 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

LayerZero Labs engaged OtterSec to assess the `console-oft`, `console-transfer-hook`, and the upgraded `solana-oapp` programs. This assessment was conducted between June 16th and June 26th, 2026. Follow-up reviews were conducted on July 14th, July 15th, July 20th, and July 26th, 2026. The latest audit was performed between September 29th and October 10th, 2026.

For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 13 findings throughout this audit engagement.

In particular, we identified a vulnerability in the default admin setup logic, which blindly overwrites the admin, so if init-if-needed constraint is utilized on the state account, a second initialize may replace the current admin, bypassing the intended two-step transfer mechanism (OS-LCS-ADV-00). We also highlighted a semantic mismatch between the EVM and Solana implementations. On EVM, a missing peer is treated as invalid, resulting in a failure. In contrast, on Solana it may still appear as a valid but empty result, potentially misleading developers into believing a peer is configured when it is not (OS-LCS-ADV-01). Furthermore, receive planning can fail for every inbound message when a pubkey policy is configured, and the live ACL configuration uses the default gate, because the first planning step omits an account that the second step later requires (OS-LCS-ADV-02).

We also suggested reading the live mint hook directly within LzReceiveTypesInfo in console-oft (OS-LCS-SUG-00), and updating the solana-evm-parity-analysis documentation with the correct type for fee-amount-ld (OS-LCS-SUG-01). We further advised implementing a consistent policy across all admin setters in console-oft to reject idempotent updates (OS-LCS-SUG-02), and strengthening compile-time and runtime validation by enforcing expected field types, rejecting ambiguous Context types, and verifying stored role values in addition to PDA-derived role seeds (OS-LCS-SUG-05).

Additionally, we recommended uniquifying injected role-member fields and detecting cross-domain entrypoint name collisions to improve composability and eliminate confusing compile-time collisions (OS-LCS-SUG-06), and reading the existing source/destination transfer-hook account extension and requiring transferring to be set (OS-LCS-SUG-03).

© 2026 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/LayerZero-Labs/audit-external. This audit was performed against commit f714fec.

Follow-up audits were performed against 85a3a47, 5d94aa2, 5cc7cac, c0197e4, and f78811f.

Finally, the codebase was published in the Git repository at https://github.com/LayerZero-Labs/monorepo-external, against commit d88ec63.

Brief descriptions of the programs are as follows:

Name	Description
console-oft	A Solana/SVM LayerZero OFT program that handles cross-chain token receive execution, including peer/rate-limit validation, token minting or escrow release, and optional compose-message handling. It also manages Token-2022 transfer-hook account discovery and validation so off-chain executors can supply the correct accounts for `lz_receive`.
console-transfer-hook	A Token-2022 transfer-hook compliance program that enforces mint-scoped pause, blacklist/whitelist, source bypass, and PermanentDelegate recovery rules on token transfers.
oapp	It standardizes how apps interact with LayerZero by enforcing a common instruction interface, account discovery model, and RBAC-based admin controls. It enables consistent cross-chain messaging—handling peer configuration, message execution, and SDK interoperability-so all OApps behave uniformly without custom integration logic.

Below is the list of all contracts that were within the scope:

Console OFT App

1. `programs/console_oft/src/lib.rs`
2. `programs/console_oft/src/console_oft_info.rs`
3. `programs/console_oft/src/events.rs`
4. `programs/console_oft/src/errors.rs`
5. `programs/console_oft/src/seeds.rs`

© 2026 Otter Audits LLC. All Rights Reserved.

6. `programs/console_oft/src/codec/mod.rs`
7. `programs/console_oft/src/codec/msg.rs`
8. `programs/console_oft/src/codec/compose_msg.rs`
9. `programs/console_oft/src/state/mod.rs`
10. `programs/console_oft/src/state/oft.rs`
11. `programs/console_oft/src/state/rate_limit.rs`
12. `programs/console_oft/src/helpers/mod.rs`
13. `programs/console_oft/src/helpers/token2022.rs`
14. `programs/console_oft/src/helpers/transfer_hook_accounts.rs`
15. `programs/console_oft/src/helpers/transfer_hook_plan.rs`
16. `programs/console_oft/src/helpers/receive_accounts.rs`
17. `programs/console_oft/src/helpers/pubkey_policy.rs`
18. `programs/console_oft/src/helpers/credit_redirect.rs`
19. `programs/console_oft/src/instructions/mod.rs`
20. `programs/console_oft/src/instructions/init_console_oft_info.rs`
21. `programs/console_oft/src/instructions/init_console_oft.rs`
22. `programs/console_oft/src/instructions/send.rs`
23. `programs/console_oft/src/instructions/quote_send.rs`
24. `programs/console_oft/src/instructions/quote_oft.rs`
25. `programs/console_oft/src/instructions/oft_store_view.rs`
26. `programs/console_oft/src/instructions/lz_receive.rs`
27. `programs/console_oft/src/instructions/lz_receive_types_info.rs`
28. `programs/console_oft/src/instructions/lz_receive_types_v3.rs`
29. `programs/console_oft/src/instructions/sync_transfer_hook_program.rs`
30. `programs/console_oft/src/instructions/sync_acl_gating_program.rs`
31. `programs/console_oft/src/instructions/set_token_manager.rs`
32. `programs/console_oft/src/instructions/set_credit_redirect_config.rs`
33. `programs/console_oft/src/instructions/set_pubkey_policy.rs`
34. `programs/console_oft/src/instructions/fee_config/mod.rs`
35. `programs/console_oft/src/instructions/fee_config/fee_config_view.rs`
36. `programs/console_oft/src/instructions/fee_config/set_default_fee_bps.rs`
37. `programs/console_oft/src/instructions/fee_config/set_fee_bps.rs`
38. `programs/console_oft/src/instructions/fee_handler/mod.rs`
39. `programs/console_oft/src/instructions/fee_handler/set_fee_deposit.rs`

© 2026 Otter Audits LLC. All Rights Reserved.

40. `programs/console_oft/src/instructions/pause/mod.rs`
41. `programs/console_oft/src/instructions/pause/pause_config_view.rs`
42. `programs/console_oft/src/instructions/pause/set_default_paused.rs`
43. `programs/console_oft/src/instructions/pause/set_paused.rs`
44. `programs/console_oft/src/instructions/rate_limiter/mod.rs`
45. `programs/console_oft/src/instructions/rate_limiter/rate_limits.rs`
46. `programs/console_oft/src/instructions/rate_limiter/get_rate_limit_usages.rs`
47. `programs/console_oft/src/instructions/rate_limiter/is_rate_limit_address_exempt.rs`
48. `programs/console_oft/src/instructions/rate_limiter/set_rate_limit_global_config.rs`
49. `programs/console_oft/src/instructions/rate_limiter/set_rate_limit_config.rs`
50. `programs/console_oft/src/instructions/rate_limiter/set_rate_limit_state.rs`
51. `programs/console_oft/src/instructions/rate_limiter/set_rate_limit_address_exemption.rs`
52. `programs/console_oft/src/instructions/rate_limiter/checkpoint_rate_limit.rs`
53. `Cargo.toml`
54. `programs/console_oft/Cargo.toml`

OApp

1. `src/lib.rs`
2. `src/common.rs`
3. `src/endpoint_cpi.rs`
4. `src/oapp_info.rs`
5. `src/lz_receive_types_v3.rs`
6. `src/lz_compose_types_v3.rs`
7. `src/lz_plan.rs`
8. `src/options.rs`
9. `src/types.rs`
10. `src/errors.rs`
11. `src/events.rs`
12. `src/utils.rs`
13. `macros/src/lib.rs`
14. `macros/src/oapp.rs`
15. `macros/src/composer.rs`
16. `macros/src/state_types.rs`
17. `macros/src/instructions/oapp/mod.rs`

© 2026 Otter Audits LLC. All Rights Reserved.

18. `macros/src/instructions/oapp/set_peer.rs`
19. `macros/src/instructions/oapp/get_peer.rs`
20. `macros/src/instructions/oapp/init_enforced_options.rs`
21. `macros/src/instructions/oapp/set_enforced_options.rs`
22. `macros/src/instructions/oapp/get_enforced_options.rs`
23. `macros/src/instructions/oapp/next_nonce.rs`
24. `macros/src/instructions/oapp/is_compose_msg_sender.rs`
25. `macros/src/instructions/oapp/lz_receive.rs`
26. `macros/src/instructions/oapp/lz_receive_types_info.rs`
27. `macros/src/instructions/oapp/lz_receive_types_v3.rs`
28. `macros/src/instructions/oapp/clear.rs`
29. `macros/src/instructions/oapp/skip.rs`
30. `macros/src/instructions/oapp/burn.rs`
31. `macros/src/instructions/oapp/nilify.rs`
32. `macros/src/instructions/oapp/accept_default_admin_transfer.rs`
33. `macros/src/instructions/composer/mod.rs`
34. `macros/src/instructions/composer/lz_compose.rs`
35. `macros/src/instructions/composer/lz_compose_types_info.rs`
36. `macros/src/instructions/composer/lz_compose_types_v3.rs`
37. `macros/src/instructions/mod.rs`
38. `Cargo.toml`
39. `macros/Cargo.toml`

Token Manager

1. `programs/token_manager/src/lib.rs`
2. `programs/token_manager/src/state.rs`
3. `programs/token_manager/src/types.rs`
4. `programs/token_manager/src/authority.rs`
5. `programs/token_manager/src/errors.rs`
6. `programs/token_manager/src/events.rs`
7. `programs/token_manager/src/instructions/mod.rs`
8. `programs/token_manager/src/instructions/root/`
9. `programs/token_manager/src/instructions/root/init_token_manager.rs`
10. `programs/token_manager/src/instructions/root/rbac_default_admin.rs`

© 2026 Otter Audits LLC. All Rights Reserved.

11. `programs/token_manager/src/instructions/native_authority/`
12. `programs/token_manager/src/instructions/native_authority/set_native_authority.rs`
13. `programs/token_manager/src/instructions/transfer_hook/set_transfer_hook_policy.rs`
14. `programs/token_manager/src/instructions/permissioned_burn/burn.rs`
15. `programs/token_manager/src/instructions/permanent_delegate/pd_burn.rs`
16. `programs/token_manager/src/instructions/permanent_delegate/pd_recover.rs`
17. `programs/token_manager/src/instructions/pause/set_paused.rs`
18. `programs/token_manager/src/instructions/mint/mint_to.rs`
19. `programs/token_manager/src/instructions/acl/init_acl.rs`
20. `programs/token_manager/src/instructions/acl/gate.rs`
21. `programs/token_manager/src/instructions/acl/freeze_token_account.rs`
22. `programs/token_manager/src/instructions/acl/thaw_token_account.rs`
23. `programs/token_manager/src/instructions/acl/manage_acl.rs`
24. `programs/token_manager/src/instructions/acl/helpers.rs`
25. `programs/token_manager/src/instructions/get_version.rs`
26. `programs/token_manager/src/instructions/mod.rs`
27. `programs/token_manager/src/instructions/*/mod.rs`
28. `interface/src/lib.rs`
29. `interface/src/state.rs`
30. `interface/src/seeds.rs`
31. `interface/src/burn.rs`
32. `interface/src/version.rs`
33. `interface/src/acl/mod.rs`
34. `interface/src/acl/manifest.rs`
35. `interface/src/acl/thaw.rs`
36. `Cargo.toml`
37. `interface/Cargo.toml`
38. `programs/token_manager/Cargo.toml`

Pubkey Policy

1. `programs/pubkey_policy/src/lib.rs`
2. `programs/pubkey_policy/src/types.rs`
3. `programs/pubkey_policy/src/errors.rs`
4. `programs/pubkey_policy/src/events.rs`

© 2026 Otter Audits LLC. All Rights Reserved.

5. `programs/pubkey_policy/src/instructions/init_pubkey_policy.rs`
6. `programs/pubkey_policy/src/instructions/membership.rs`
7. `programs/pubkey_policy/src/instructions/set_blacklisted.rs`
8. `programs/pubkey_policy/src/instructions/set_whitelisted.rs`
9. `programs/pubkey_policy/src/instructions/set_mode.rs`
10. `programs/pubkey_policy/src/instructions/get_version.rs`
11. `programs/pubkey_policy/src/instructions/mod.rs`
12. `interface/src/lib.rs`
13. `interface/src/state.rs`
14. `interface/src/seeds.rs`
15. `interface/src/pubkey_policy.rs`
16. `interface/src/error.rs`
17. `interface/src/version.rs`
18. `Cargo.toml`
19. `interface/Cargo.toml`
20. `programs/pubkey_policy/Cargo.toml`

Pubkey Policy Hook

1. `programs/pubkey_policy_hook/src/lib.rs`
2. `programs/pubkey_policy_hook/src/state.rs`
3. `programs/pubkey_policy_hook/src/errors.rs`
4. `programs/pubkey_policy_hook/src/events.rs`
5. `programs/pubkey_policy_hook/src/instructions/init_transfer_hook.rs`
6. `programs/pubkey_policy_hook/src/instructions/transfer_hook.rs`
7. `programs/pubkey_policy/src/instructions/mod.rs`
8. `Cargo.toml`
9. `interface/Cargo.toml`
10. `programs/pubkey_policy_hook/Cargo.toml`

Program ID

1. `lib/src/lib.rs`
2. `lib/Cargo.toml`
3. `Cargo.toml`

© 2026 Otter Audits LLC. All Rights Reserved.

Heap Allocator

1. `lib/src/lib.rs`
2. `lib/Cargo.toml`
3. `Cargo.toml`

anchor-trait

1. `src/lib.rs`
2. `src/overrides.rs`
3. `src/generation.rs`
4. `src/assembly.rs`
5. `Cargo.toml`

rbac

1. `src/lib.rs`
2. `src/traits.rs`
3. `src/types.rs`
4. `src/errors.rs`
5. `src/events.rs`
6. `macros/src/lib.rs`
7. `macros/src/rbac.rs`
8. `macros/src/state_types.rs`
9. `macros/src/role_type.rs`
10. `macros/src/only_role.rs`
11. `macros/src/init_default_admin.rs`
12. `macros/src/setup_default_admin.rs`
13. `macros/src/utils.rs`
14. `macros/src/instructions/mod.rs`
15. `macros/src/instructions/grant_role.rs`
16. `macros/src/instructions/revoke_role.rs`
17. `macros/src/instructions/renounce_role.rs`
18. `macros/src/instructions/begin_default_admin_transfer.rs`
19. `macros/src/instructions/accept_default_admin_transfer.rs`
20. `macros/src/instructions/get_default_admin.rs`
21. `Cargo.toml`
22. `macros/Cargo.toml`

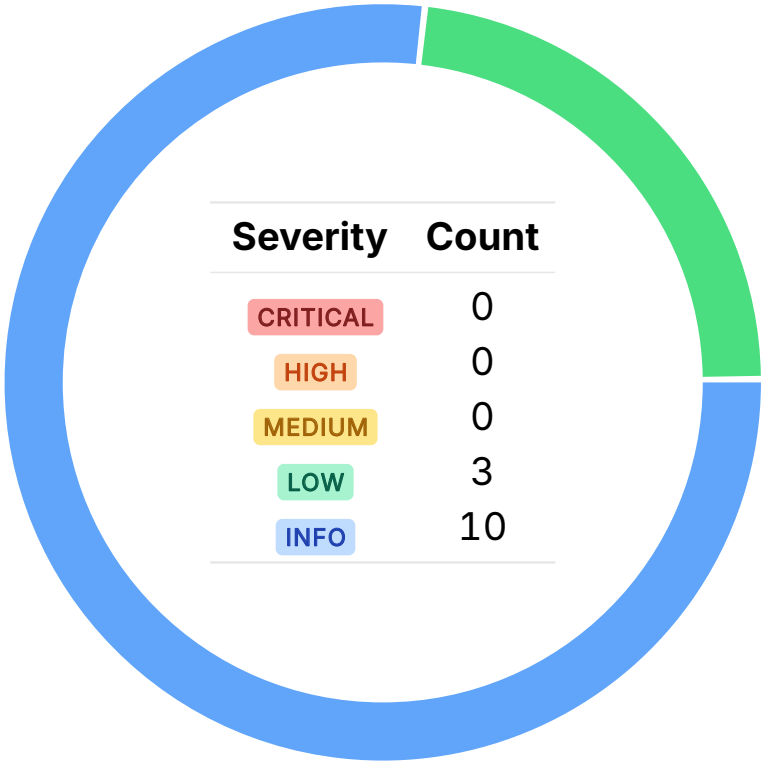
© 2026 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 13 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.

Rating criteria can be found in Appendix A.



Severity	Count
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	3
INFO	10

© 2026 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-LCS-ADV-00	LOW	RESOLVED	`setup_default_admin!` blindly overwrites the admin, so if `init_if_needed` is utilized on the state account, a second initialize may replace the current admin, bypassing the intended two-step transfer mechanism.
OS-LCS-ADV-01	LOW	RESOLVED	EVM treats a zero peer (`bytes32(0)`) as unset and reverts, while Solana may return `Some([0; 32])` , falsely implying a valid peer on utilizing `is_some` to check peer existence.
OS-LCS-ADV-02	LOW	RESOLVED	`lz_receive_types_info` can omit `acl_thaw_metas` when the cached ACL gate is `Pubkey::default()` , even though `lz_receive_types_v3` later reads the live `MintConfig` and requires the thaw manifest. As a result, inbound receive planning can fail when a policy is configured, but the live ACL configuration uses the default gate.

© 2026 Otter Audits LLC. All Rights Reserved.

Unsafe Admin Reinitialization LOW OS-LCS-ADV-00

Description

`setup_default_admin!` unconditionally writes `current_default_admin` and clears `pending_default_admin` without first checking whether the RBAC state has already been initialized. This is safe only if the integrator's initialize accounts are strictly protected with Anchor `init`. However, if `init_if_needed` constraint is utilized, a second initialization call may pass with a different admin because the `admin_role_member` PDA is derived utilizing the new admin, producing a fresh role-member account rather than colliding with the existing one. As a result, the live default admin may be overwritten outside the intended two-step admin transfer flow, bypassing the safety guarantees of `begin` / `accept` admin transfer flow.

Remediation

Require that the state's `current_default_admin` is `Pubkey::default()` before assignment, reject re-initialization, by adding `require!(state.current_default_admin() == &Pubkey::default(), AlreadyInitialized)` before `set_current_default_admin`.

Patch

Resolved in f1c4df2.

© 2026 Otter Audits LLC. All Rights Reserved.

EVM–Solana Peer Existence Semantics Mismatch LOW OS-LCS-ADV-01

Description

There is a semantic inconsistency in how an unset peer is represented across chains. On EVM, `_getPeerOrRevert` treats `bytes32(0)` as unset and explicitly reverts, so zero is never considered a valid peer. On Solana, `get_peer` returns `Option<[u8; 32]>`, but if a peer account exists with address `[0; 32]`, it returns `Some([0; 32])` instead of signaling absence. Thus, `is_some` may incorrectly imply a valid peer exists, misleading developers if they utilize `is_some` to check peer existence on Solana.

>_ *oapp-app/contracts/solana/macros/src/instructions/oapp /get_peer.rs* RUST

```rust
impl StdGetPeer<'_> {
    pub fn #handler_ident(ctx: &Context<Self>, _eid: &#params_type) -> Result<#return_type> {
        let peer = ::oapp::utils::try_load_account::<OAppPeer>(&ctx.accounts.peer)?;
        Ok(peer.map(|p| p.address))
    }
}
```

Remediation

Update the Solana implementation such that it maps `[0; 32]` → `None`, ensuring zeroed peer is consistently represented across environments. Alternatively, document the divergence clearly and recommend a non-zero check at the call site.

Patch

Resolved in b2fe42c.

© 2026 Otter Audits LLC. All Rights Reserved.

Default Gate Omits Thaw Metas LOW

OS-LCS-ADV-02

Description

`lz_receive_types_info` uses the cached `OFTStore.acl_gating_program` as a sentinel and only returns `acl_thaw_metas` when that cached value is non-default. However, `lz_receive_types_v3` does not use the cache as the source of truth. It reads the live Token ACL `MintConfig`, obtains its `gating_program`, and requires the corresponding thaw manifest account. If a pubkey policy is set and the mint's live `MintConfig.gating_program` is legitimately `Pubkey::default()`, `lz_receive_types_info` omits `acl_thaw_metas`, while `lz_receive_types_v3` still expects it. That causes receive planning to fail with `InvalidAclThawMetas` for every inbound message, and `sync_acl_gating_program` cannot repair the state because syncing preserves the same default key.

Remediation

Include `acl_thaw_metas` whenever a pubkey policy is set, or cache `acl_gating_program` as `Option<Pubkey>` so "no ACL config" and "default gate configured" are distinct.

Patch

Resolved in 4d6e6826.

© 2026 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings identified during our audit. While these findings do not pose an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-LCS-SUG-00	A stale cached `transfer_hook_program` may cause `lz_receive_types_info` to return obsolete hook accounts, resulting in `lz_receive_types_v2` or `lz_receive` rejecting until `sync_transfer_hook_program` is called.
OS-LCS-SUG-01	The parity markdown has a documentation error, where it lists `OFTFeeDetail.fee_amount_ld` as `i64`, while the Solana code uses `i128`.
OS-LCS-SUG-02	Some admin setters reject no-op updates, while others allow the same value to be written again, resulting in redundant writes and duplicate events.
OS-LCS-SUG-03	`transfer_hook` lacks a `TransferHookAccount.transferring == true` check, so direct standalone calls are indistinguishable from real Token-2022 transfer-hook invocations.
OS-LCS-SUG-04	The framework should correct misleading serialization/layout documentation, explicitly document critical security invariants such as peer validation in `lz_receive`, and clarify or restrict macro-driven handler relocation semantics that may alter attribute behavior.
OS-LCS-SUG-05	The codebase should strengthen compile-time and runtime validation by enforcing expected field types, rejecting ambiguous `Context` types, and verifying stored role values in addition to PDA-derived role seeds.

© 2026 Otter Audits LLC. All Rights Reserved.

OS-LCS-SUG-06	Suggestions to prevent duplicate-generated fields and entrypoint names by uniquifying injected role-member fields and detecting cross-domain entrypoint name collisions to improve composability and eliminate confusing compile-time collisions.
OS-LCS-SUG-07	`system_program` is unnecessary in `revoke_role` and `renounce_role` because the `close` does not invoke it.
OS-LCS-SUG-08	Stacking multiple instruction attributes within a single function causes only the first domain to process it. In contrast, the others are silently ignored, resulting in missing handlers without any compile-time error.
OS-LCS-SUG-09	The configuration setters and permissionless sync instructions emit events even when no stored value changes, resulting in emission of unnecessary events and cluttering the event logs.

© 2026 Otter Audits LLC. All Rights Reserved.

Stale Hook Cache Blocks Inbound Planning

OS-LCS-SUG-00

Description

`lz_receive_types_info` relies on the cached `OFTStore.transfer_hook_program` to return the hook program and extra-account-meta-list PDA needed for `lz_receive_types_v2` planning. If the Token-2022 mint's transfer-hook extension is repointed externally, that cache becomes stale until `sync_transfer_hook_program` is called. During that stale window, `lz_receive_types_v2` validates the supplied hook account against the mint's live hook and rejects with `InvalidTransferHookProgram`. The runtime `lz_receive` path will likewise reject in `split_and_validate_hook_accounts`. This does not lose funds or corrupt state, but it will halt inbound receive planning/execution until any caller performs the permissionless sync transaction.

Remediation

Read the live mint hook directly in `lz_receive_types_info`, which removes the stale-cache dependency and eliminates both the cached field and the sync instruction.

Patch

LayerZero acknowledged this issue and stated that the cached `transfer_hook_program` behavior is intentional due to the current `lz_receive_types_v2` design. Specifically, `lz_receive_types_info` only works with the two PDA accounts it already receives, so the mint account cannot be added to read the live hook directly. This constraint necessitates the `sync_transfer_hook_program` instruction.

© 2026 Otter Audits LLC. All Rights Reserved.

OFTFeeDetail Type Mismatch in Docs

OS-LCS-SUG-01

Description

`solana-evm-parity-analysis.md` incorrectly documents Solana `OFTFeeDetail.fee_amount_ld` as `i64`, while the actual `types` definition uses `i128`. `i128` is a closer Solana-side approximation to EVM `int256`.

>_ apps/oft-app/contracts/solana/oft/src/types.rs RUST

```rust
/// Struct representing OFT fee details.
/// Future-proof mechanism to provide a standardized way to communicate fees to things like a
    ↪ UI.
/// EVM: OFTFeeDetail
#[derive(Clone, AnchorSerialize, AnchorDeserialize)]
pub struct OFTFeeDetail {
    /// Amount of the fee in local decimals. Negative values represent rebates.
    pub fee_amount_ld: i128,
    /// Description of the fee.
    pub description: String,
}
```

Remediation

Update the documentation's type table.

Patch

LayerZero acknowledged this issue and stated that they would update the parity documentation to reflect `fee_amount_ld` as `i128`.

© 2026 Otter Audits LLC. All Rights Reserved.

Inconsistent Idempotency Across Admin Setters

OS-LCS-SUG-02

Description

In the current implementation, setter idempotency is inconsistent as `set_default_paused` and `set_rate_limit_address_exemption` reject no-op updates, while `set_paused`, `set_fee_bps`, `set_default_fee_bps`, `set_rate_limit_config`, and `set_rate_limit_state` allow writing the same effective value again. This creates redundant state writes and duplicate events, reducing overall efficiency and readability.

Remediation

Implement a consistent policy across all admin setters to reject idempotent updates.

Patch

LayerZero acknowledged this issue and stated that setters without idempotency guards are intentionally preserved to maintain consistency with EVM behavior.

© 2026 Otter Audits LLC. All Rights Reserved.

Missing Transfer Invocation Guard

OS-LCS-SUG-03

Description

The `transfer_hook` instruction is publicly callable through the SPL Transfer Hook Execute discriminator, but it does not verify the Token-2022 `TransferHookAccount.transferring` flag on the source or destination token account. Without that check, the hook cannot distinguish an authentic Token-2022 transfer-hook invocation from a standalone direct call into the hook instruction.

This is harmless in the current implementation because the hook is read-only, and a direct call only evaluates checks without mutating state or moving funds. However, if future hook logic adds side effects, the missing `transferring == true` guard becomes load-bearing

Remediation

Read the existing source/destination transfer-hook account extension and require `transferring` to be set.

Patch

LayerZero acknowledged this issue and stated that they had previously considered adding the `transferring` check, but omitted it because the hook is currently read-only and a direct standalone call is a no-op. Nonetheless, they agreed that the check would provide useful hardening, especially if the hook gains side effects in the future.

© 2026 Otter Audits LLC. All Rights Reserved.

Inconsistencies in Documentation and Macro Semantics OS-LCS-SUG-04

Description

1. The comment in `EnforcedOptions::space()` misrepresents the actual Borsh serialization order. It describes the borsh field layout as `discriminator` (8) + `bump` (1) + `vec_len_prefix` (4) + `options_data`, but in Borsh, fields are serialized strictly in declaration order, so `options: Vec<u8>` is encoded first (4-byte length prefix + data), followed by `bump: u8`. The comment instead suggests `bump` comes before the vector, which implies incorrect field offsets. Thus, while the total size calculation is still correct, the documented field offsets are wrong.

>_ apps/oapp-app/contracts/solana/macros/src/state_types.rs RUST

```rust
/// Generates the default `EnforcedOptions` account struct.
pub fn generate_enforced_options() -> TokenStream {
    quote! {
        #[account]
        pub struct EnforcedOptions {
            pub options: Vec<u8>,
            pub bump: u8,
        }

        impl EnforcedOptions {
            /// Returns the total account space required for the given options length.
            ///
            /// Layout: discriminator (8) + bump (1) + vec_len_prefix (4) + options_data
            pub fn space(options_len: usize) -> usize {
                8 + 1 + 4 + options_len
            }
        }
    }
}
```

2. The Solana `lz_receive` instruction deliberately provides no default implementation, pushing full responsibility for message handling—including security checks—onto the developer. Unlike the EVM counterpart (`OAppReceiver::lzReceive`), which enforces that the caller is the endpoint and that the source sender matches a configured trusted peer before invoking `_lzReceive`, the Solana version does not embed any such validation. As a result, developers porting logic from EVM may assume these guarantees exist implicitly and omit peer verification, unintentionally allowing messages from untrusted sources to be processed.

>_ oapp-app/contracts/solana/macros/src/instructions/oapp/lz_receive.rs RUST

```rust
//! Code generator for the `lz_receive` instruction.
//!
//! This instruction has no default implementation — users MUST provide
//! a custom handler named `lz_receive` with `#[oapp_instruction]`.

use super::CodegenContext;
use anchor_trait::InstructionSpec;
use quote::quote;

pub(super) fn spec() -> InstructionSpec<CodegenContext> {
    InstructionSpec {
```

© 2026 Otter Audits LLC. All Rights Reserved.

```
            params_type: Some(quote!(::oapp::types::LzReceiveParams)),
            return_type: quote!(()),
            is_view: false,
            default_impl: None,
        }
    }
```

3. `wrap_handler` only strips the override attribute and preserves all other function-level attributes when relocating the handler into a generated wrapper module. This implies attributes such as `#[tracing::instrument]`, `#[inline]`, or custom procedural macros are now evaluated in a different lexical and module context (`mod __... use super::*; ...`) instead of the original `#[program]` scope. Since some attributes rely on surrounding items, visibility, or macro expansion order, this relocation may subtly change their behavior or result in them malfunctioning without obvious errors.

>_ contracts/common/framework/solana/anchor-trait/src/overrides.rs RUST

```rust
fn wrap_handler(&mut self, target: Ix, fn_item: &ItemFn) -> syn::Result<HandlerMeta> {
    let context_type = infer_context_type(fn_item)?;
    let fn_generics = fn_item.sig.generics.clone();
    // Generate a deterministic module name: `__<attr>__<instruction>_handler`.
    let wrapper_mod =
        format_ident!("__{}__{}_handler", self.config.attr_name, target.canonical_name());
    // Clone the function without the instruction attribute.
    let mut wrapped_fn = fn_item.clone();
    wrapped_fn.attrs.retain(|attr| !is_override_attr(attr, self.config));
    [...]
}
```

4. The setup documentation grants the `Burner` role to the `OFTStore` PDA, but the `PermissionedBurn` path in `send` invokes `TokenManager` with `oft_escrow` as both authority and `source_authority`. As a result, deployments that follow the `README` configure the role on the wrong principal, causing sends on `PermissionedBurn`-enabled mints to fail during the burn authorization check.

Remediation

1. Update the comment to reflect the correct Borsh serialization order, or reorder the structure fields to put `bump` first if the documented layout is preferred.
2. Explicitly document that `lz_receive` handlers must validate (`src_eid`, `sender`) against configured peers, aligning with EVM semantics. Providing a helper such as `require_peer` will further reduce the risk of inconsistent or incorrect implementations.
3. Either disallow foreign attributes on override handlers (emit `compile_error!`) or explicitly document the relocation and add a snapshot test to assert the expected behavior.
4. Update the `README` to grant the Burner role to the `oft_escrow` PDA used by send, and replace the stale ACL module link.

Patch

1. Issue #1 resolved in b2fe42c.
2. Issue #2 resolved in b2fe42c.
3. Issue #3 was acknowledged by LayerZero.

© 2026 Otter Audits LLC. All Rights Reserved.

Insufficient Validation and Type Safety OS-LCS-SUG-05

Description

1. `validate_named_struct` only verifies that required field names exist, but does not enforce their types, which weakens the guarantees of the `DefaultAdmin` derive. As a result, a structure may define `current_default_admin` or `pending_default_admin` with an incorrect type (such as `u64`) and still pass validation. The macro will then generate an `impl DefaultAdmin` that assumes these fields are `Pubkey`, resulting in confusing type errors later during compilation.

>_ *contracts/common/utils/solana/rbac/macros/src/utils.rs* RUST

```rust
pub fn validate_named_struct(
    input: &DeriveInput,
    derive_name: &str,
    required_fields: &[&str],
) -> syn::Result<()> {
    let name = &input.ident;

    let Data::Struct(data) = &input.data else {
        return Err(syn::Error::new_spanned(
            name,
            format!("{derive_name} can only be derived for structs"),
        ));
    };
    let Fields::Named(fields) = &data.fields else {
        return Err(syn::Error::new_spanned(
            name,
            format!("{derive_name} can only be derived for structs with named fields"),
        ));
    };

    let missing: Vec<_> = required_fields
        .iter()
        .copied()
        .filter(|req| {
            !fields
                .named
                .iter()
                .any(|f| f.ident.as_ref().is_some_and(|id| id == req))
        })
        .collect();
    [...]
    Ok(())
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

2. The current check in `is_context_type` only verifies that the last segment of the type path is named `Context`, without ensuring it actually refers to Anchor's Context type. This allows cases where a user imports or defines another Context (such as `use foo::Context`) that shadows `anchor_lang::context::Context`, and the validation still passes. As a result, `infer_context_type` may accept a structurally similar but semantically incorrect type, rewulting in codegen to operate on the wrong abstraction, which may produce confusing compile-time errors.

>_ *contracts/common/framework/solana/anchor-trait/src/overrides.rs* RUST

```rust
/// Returns `true` if the type's last path segment is `Context`.
fn is_context_type(ty: &Type) -> bool {
    matches!(ty, Type::Path(tp) if tp.path.segments.last().is_some_and(|s| s.ident ==
        ↪  "Context"))
}
```

3. The `RoleMember` PDA derivation in `#[only_role]` macro relies on `RoleType::seed(role)` producing a unique byte representation for each role. If a developer implements a non-bijective mapping (a manual `Into<u8>` with collisions, or `#[repr(u16)]` with a truncating cast), two distinct roles may resolve to the same seed. In that case, the PDA for role `X` and role `Y` would collide, and a valid `RoleMember` for `X` may satisfy the PDA constraint for `Y`. This breaks the authorization guarantee of `#[only_role]`, since seed validation alone would be insufficient.

>_ *contracts/common/utils/solana/rbac/macros/src/only_role.rs* RUST

```rust
pub fn only_role_impl(attr: TokenStream, item: TokenStream) -> syn::Result<TokenStream> {
    [...]
    let fields = utils::named_fields_mut(&mut accounts_struct)?;
    let new_field = Field::parse_named.parse2(quote! {
        /// Role membership proof — authority must have the required role.
        #[account(
            seeds = [
                ::rbac::ROLE_MEMBER_SEED,
                #state_field.key().as_ref(),
                &::rbac::traits::RoleType::seed(&(#role)),
                #authority_field.key().as_ref(),
            ],
            bump = role_member.bump,
        )]
        pub role_member: Account<'info, RoleMember>
    })?;
    fields.named.push(new_field);
    Ok(quote!(#accounts_struct))
}
```

© 2026 Otter Audits LLC. All Rights Reserved.

Remediation

1. Extend `validate_named_struct` to accept a `&[(&str, &str)]` of (field name, expected type) pairs, compare each field's `ty` token stream against the expected type, and emit a domain-aware `compile_error!` at the derive site on mismatch.
2. Match the full canonical path (`anchor_lang::context::Context`), with allowances for aliases, or explicitly reject shadowed Context types to ensure correctness. Alternatively, require a fully-qualified path in handler signatures and document it.
3. Add an explicit `role_member.role == #role` check in the generated authorization logic to verify that the role stored in the account matches the requested role, rather than relying solely on the seed byte. Additionally, document or enforce via tests the bijectivity of `RoleType: Into<u8>` .

Patch

1. Issue #1 was acknowledged by LayerZero.
2. Issue #2 was acknowledged by LayerZero.
3. Issue #3 resolved in f1c4df2.

© 2026 Otter Audits LLC. All Rights Reserved.

Code Duplication

OS-LCS-SUG-06

Description

1. The `#[only_role]` macro always injects a field named `role_member`, which creates a structural limitation when multiple role checks are needed on the same instruction. If a developer stacks multiple `#[only_role]` attributes (such as requiring both `Pauser` and `FeeAdmin` roles), each macro expansion attempts to insert a field with the same name, creating a duplicate field definition and resulting in compile-time error. This prevents legitimate multi-role authorization patterns from being expressed at the account level.

   >_ *contracts/common/utils/solana/rbac/macros/src/only_role.rs* RUST

   ```rust
   pub fn only_role_impl(attr: TokenStream, item: TokenStream) -> syn::Result<TokenStream> {
       [...]
       let fields = utils::named_fields_mut(&mut accounts_struct)?;
       let new_field = Field::parse_named.parse2(quote! {
           /// Role membership proof — authority must have the required role.
           #[account(
               seeds = [
                   ::rbac::ROLE_MEMBER_SEED,
                   #state_field.key().as_ref(),
                   &::rbac::traits::RoleType::seed(&(#role)),
                   #authority_field.key().as_ref(),
               ],
               bump = role_member.bump,
           )]
           pub role_member: Account<'info, RoleMember>
       })?;
       fields.named.push(new_field);
       Ok(quote!(#accounts_struct))
   }
   ```

2. `InstructionSet::canonical_name` is scoped per domain, so different macro domains (such as `#[oapp]` and `#[composer]`) may independently define instructions with the same function name (such as `lz_compose`). During assembly, both domains generate entrypoints into the same `#[program]` module, resulting in duplicate function definitions. While Anchor eventually rejects this with a duplicate symbol error, the diagnostic originates from the fully expanded, macro-generated code, lacking context about which domains contributed the conflicting definitions. Rendering the root cause non-obvious and difficult to trace.

   >_ *contracts/common/framework/solana/anchor-trait/src/lib.rs* RUST

   ```rust
   /// Trait implemented by each macro crate's instruction name enum.
   ///
   /// Generated by [`declare_instructions!`] or implemented manually.
   pub trait InstructionSet: Copy + Ord + 'static {
       /// The domain-specific codegen context type (e.g. `CodegenContext`).
       type Ctx;

       /// All known instruction variants in deterministic order.
       fn all() -> &'static [Self];
       /// Parses an instruction name from an attribute argument ident.
       fn from_ident(ident: &syn::Ident) -> Option<Self>;
       /// The canonical function name for this instruction (e.g. `"set_peer"`).
   ```

© 2026 Otter Audits LLC. All Rights Reserved.

```
    fn canonical_name(&self) -> &'static str;
    /// Returns the code generation spec for this instruction variant.
    fn spec(self, ctx: &Self::Ctx) -> InstructionSpec<Self::Ctx>;
}
```

Remediation

1. Derive a unique field name per role (`role_member_pauser` or `role_member_fee_admin`) by hashing the role expression, allowing multiple role constraints to coexist within the same structure, or accept an optional `as = ident` argument so the developer may name the injected member explicitly.
2. Detect duplicate entrypoint identifiers in `Assembly::assemble` and emit a clear, domain-aware compile-time error before handing off to Anchor.

Patch

1. Issue #1 was acknowledged by LayerZero.
2. Issue #2 was acknowledged by LayerZero.

© 2026 Otter Audits LLC. All Rights Reserved.

Unnecessary System Program Account in RBAC OS-LCS-SUG-07

Description

The `system_program` account in `revoke_role` and `renounce_role` is redundant because both instructions utilize Anchor's `close = receiver` constraint, which directly handles lamport transfer and account deallocation without performing any CPI to the `System` program. Thus, the `system_program` field is redundant in the accounts structure and introduces an unused account in every transaction.

Remediation

Remove `pub system_program: Program<'info, System>` from the `StdRevokeRole` and `StdRenounceRole` accounts structures.

Patch

Resolved in b2fe42c.

© 2026 Otter Audits LLC. All Rights Reserved.

Cross-Domain Instruction Attribute Conflict OS-LCS-SUG-08

Description

The override pipeline is domain-local, implying each macro only recognizes and processes its own `#[*_instruction]` attribute. During `wrap_handler`, only the matching attribute is stripped, while any foreign instruction attributes (such as `#[composer_instruction]` inside an `#[oapp]` pass) are preserved on the cloned function. However, `filtered_items` removes the original function from the source module after the first domain processes it. As a result, subsequent domain macros never see the original function and therefore never register their handler, even though their attribute still exists on the relocated function. Thus, stacking two domain instruction attributes for the same function (such as `#[oapp_instruction]` and `#[composer_instruction]`) silently discards the second, resulting in missing handlers without obvious errors.

>_ *contracts/common/framework/solana/anchor-trait/src/overrides.rs* RUST

```rust
fn wrap_handler(&mut self, target: Ix, fn_item: &ItemFn) -> syn::Result<HandlerMeta> {
    let context_type = infer_context_type(fn_item)?;
    let fn_generics = fn_item.sig.generics.clone();
    // Generate a deterministic module name: `__<attr>__<instruction>_handler`.
    let wrapper_mod =
        format_ident!("__{}__{}_handler", self.config.attr_name, target.canonical_name());
    // Clone the function without the instruction attribute.
    let mut wrapped_fn = fn_item.clone();
    wrapped_fn.attrs.retain(|attr| !is_override_attr(attr, self.config));
    // Emit the wrapper module so the handler lives in its own scope.
    self.wrapper_defs.extend(quote! {
        mod #wrapper_mod {
            use super::*;
            #wrapped_fn
        }
    });
    Ok(HandlerMeta { wrapper_mod, context_type, fn_generics })
}
```

Remediation

Detect the presence of foreign instruction attributes during override collection and emit a `compile_error!`, rejecting multi-domain stacking on a single function, or coordinate ordering by stripping all domain-instruction attributes uniformly and registering the handler with both domains.

Patch

This issue was acknowledged by LayerZero.

© 2026 Otter Audits LLC. All Rights Reserved.

Unnecessary Event Emissions OS-LCS-SUG-09

Description

`set_token_manager`, `set_pubkey_policy`, `set_credit_redirect_config`, `sync_transfer_hook_program`, and `sync_acl_gating_program` emit configuration events even when the stored value is already unchanged. This generates unnecessary event noise, particularly for the permissionless sync instructions, as any caller can repeatedly invoke them to produce duplicate events without causing a real state transition. Although this does not directly corrupt protocol state, it weakens event reliability for indexers, monitoring systems, and operational alerting that treat emitted events as meaningful changes.

>_ *programs/console_oft/src/lib.rs* RUST

```rust
/// Sync cached transfer_hook_program from mint extension. Permissionless.
pub fn sync_transfer_hook_program(mut ctx: Context<SyncTransferHookProgram>) -> Result<()> {
    SyncTransferHookProgram::apply(&mut ctx)
}
/// Sync cached acl_gating_program from the mint's Token ACL config. Permissionless.
pub fn sync_acl_gating_program(mut ctx: Context<SyncAclGatingProgram>) -> Result<()> {
    SyncAclGatingProgram::apply(&mut ctx)
}
// ============================== Credit admission and redirect ==============================
pub fn set_pubkey_policy(mut ctx: Context<SetPubkeyPolicy>) -> Result<()> {
    SetPubkeyPolicy::apply(&mut ctx)
}
/// EVM alignment: `setCreditRedirectConfig`. Omitting `redirect_escrow` disables redirect.
pub fn set_credit_redirect_config(mut ctx: Context<SetCreditRedirectConfig>) -> Result<()> {
    SetCreditRedirectConfig::apply(&mut ctx)
}
/// Select the TokenManager root BurnMint sends request PermissionedBurn approval from.
pub fn set_token_manager(mut ctx: Context<SetTokenManager>) -> Result<()> {
    SetTokenManager::apply(&mut ctx)
}
```

Similarly, `pause`, `unpause`, and `set_transfer_hook_policy` in `token_manager` emit events even when the requested state already matches the stored state, producing redundant configuration-change logs. However, because these functions are gated, the impact is limited and does not result in permissionless event spam.

Remediation

Compare each current value against the proposed value and return early without writing state or emitting an event when there is no effective change.

© 2026 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2026 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2026 Otter Audits LLC. All Rights Reserved.